

Chapter 8

Constants, static and enums

Object-oriented TypeScript, test first

Traffic Light



Main Street

Red and amber

This colour stays on for 2 seconds.

Next

☐ Change by itself

Today

- `const`, `readonly` and `static readonly`
- string-literal unions: "one of these values, nothing else"
- a `switch` the compiler checks for **every** value
- testing every value with a loop
- `enum` vs union vs `as const` - and why this book uses unions
- static fields and methods
- what static state does to your **tests**

Three kinds of "never changes"

You want	TypeScript	Java
a variable never reassigned	<code>const LIMIT = 3;</code>	<code>final int</code>
a field set once per object	<code>readonly name</code>	<code>private final</code>
one value for the class	<code>static readonly PREFIX</code>	<code>static final</code>

All three checked by the compiler:

```
Cannot assign to 'LIMIT' because it is a constant.  
Cannot assign to 'name' because it is a read-only property.
```

A type with four values

```
export type LightColour =  
  "red" | "red-amber" | "green" | "amber";
```

- a **string-literal union**: one of these four strings, nothing else
- at run time: just a string
- the compiler checks every place one is made:

```
TS2345 [ERROR]: Argument of type '"purple"' is not  
assignable to parameter of type 'LightColour'.
```

- the editor offers the four values as you type

nextColour, test first: red, green

```
Deno.test("after red comes red and amber", () => {  
  assertEquals(nextColour("red"), "red-amber");  
});
```

Simplest code that passes:

```
export const nextColour =  
  (colour: LightColour): LightColour => "red-amber";
```

Red again

```
Deno.test("after red and amber comes green", () => {  
  assertEquals(nextColour("red-amber"), "green");  
});
```

not ok 2 - after red and amber comes green
AssertionError: Values are not equal.

[Diff] Actual / Expected

```
- red-amber  
+ green
```

Two cases... and a surprise

```
export const nextColour = (colour: LightColour): LightColour => {  
  switch (colour) {  
    case "red":  
      return "red-amber";  
    case "red-amber":  
      return "green";  
  }  
};
```

Tests pass - but:

```
TS2366 [ERROR]: Function lacks ending return statement  
and return type does not include 'undefined'.
```

The compiler knows "green" and "amber" are possible too

Every case

```
export const nextColour = (colour: LightColour): LightColour => {  
  switch (colour) {  
    case "red":  
      return "red-amber";  
    case "red-amber":  
      return "green";  
    case "green":  
      return "amber";  
    case "amber":  
      return "red";  
  }  
};
```

- **no default** - it would hide a missing case
- needs the **return type** written - one more reason to write it

Testing every value

```
export const LIGHT_COLOURS: LightColour[] =  
  ["red", "red-amber", "green", "amber"];
```

```
Deno.test("every colour comes back after four steps", () => {  
  for (const colour of LIGHT_COLOURS) {  
    const afterFour =  
      nextColour(nextColour(nextColour(nextColour(colour))));  
    assertEquals(afterFour, colour);  
  }  
});
```

Weakness: the colours are written **twice** (type and array)

static readonly and readonly

```
export class TrafficLight {  
    // one value for the whole class  
    public static readonly STARTING_COLOUR: LightColour = "red";  
  
    private colour: LightColour = TrafficLight.STARTING_COLOUR;  
  
    // one value per object, set once  
    constructor(public readonly name: string) {}  
}
```

- static: `TrafficLight.STARTING_COLOUR` - through the **class**
- even inside the class: never `this.STARTING_COLOUR`

Record: a key for every value

```
public static readonly SECONDS: Record<LightColour, number> = {  
    "red": 5,  
    "red-amber": 2,  
    "green": 5,  
    "amber": 3,  
};
```

Leave one out:

```
TS2741 [ERROR]: Property '"red-amber"' is missing in type  
'{ red: number; green: number; amber: number; }' but  
required in type 'Record<LightColour, number>'.
```

readonly is shallow

```
TrafficLight.SECONDS = { ... };    // error: read-only  
TrafficLight.SECONDS.red = 99;    // compiles!
```

- `readonly` stops the property pointing somewhere else
- it does not freeze the object it points to
- same as Java's `final` on a list
- Chapter 9: references

Project 2: Menu Order

Menu Order

Show: Everything Vegetarian Vegan

Starter		
Tomato soup	Vegan	€8.50 Add
Garlic mushrooms	Vegetarian	€7.50 Add
Main		
Vegetable curry	Vegan	€14.50 Add
Cheese and onion pie	Vegetarian	€15.50 Add
Dessert		
Fruit salad	Vegan	€5.95 Add
Apple crumble	Vegetarian	€6.95 Add
Chocolate brownie	Vegetarian	€6.50 Add

Your order

- Garlic mushrooms - €7.50
- Vegetable curry - €14.50

Total: €22.00

Clear the order

```
public enum DietType { VEGAN, VEGETARIAN, CARNIVORE }
```

TypeScript can do this three ways

Three ways

Three ways to write a fixed set of values

What you write (TypeScript)

1. enum

```
enum DietEnum {  
  Vegan = "vegan", ... }
```

What runs (JavaScript)

a real object is made:

DietEnum.Vegan is "vegan"
but a plain "vegan" is not accepted as one

2. string-literal union (this book's choice)

```
type Diet =  
  "vegan" | "vegetarian" | "meat";
```

nothing at all - the type is erased;
values are plain strings: "vegan"

3. as const array (or object) + union from it

```
const DIETS = ["vegan", ...] as const;  
type Diet = (typeof DIETS)[number];  
use when the program needs to loop over the values
```

a plain array of strings:
["vegan", "vegetarian", "meat"]
the type is erased, the array stays

In every case the compiler rejects "vegen" - the difference is what is left when the program runs.

Way 1: enum

```
export enum DietEnum {  
  Vegan = "vegan",  
  Vegetarian = "vegetarian",  
  Meat = "meat",  
}
```

- looks like Java: `DietEnum.Vegan`
- **not just a type**: it becomes a real object in the JavaScript
- Node.js running TypeScript directly:

```
SyntaxError [ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX]:  
TypeScript enum is not supported in strip-only mode
```

Enum surprises

A plain string is not accepted - but JSON data is plain strings:

```
TS2322 [ERROR]: Type '"vegan"' is not assignable to  
type 'DietEnum'.
```

Without values, members are numbers - mapped both ways:

```
enum NumberedCourse { Starter, Main, Dessert }  
  
NumberedCourse[1]           // "Main"  
Object.keys(NumberedCourse)  
// ["0", "1", "2", "Starter", "Main", "Dessert"]
```

Way 2: a string-literal union

```
export type DietUnion = "vegan" | "vegetarian" | "meat";
```

- only a type - vanishes from the JavaScript
- values are ordinary strings
- works with `switch`, compiler checks every case
- weakness: no values at run time to loop over

Way 3: as const

```
export const DIETS = ["vegan", "vegetarian", "meat"] as const;  
export type Diet = (typeof DIETS)[number];
```

- `as const` : these exact values, never changed
- `(typeof DIETS)[number]` : the type of one element
- values at run time **and** a union – from **one** line
- the page loops over `COURSES` to build its sections

Which to use?

- a fixed set of values: a **string-literal union**
- need to list the values too (loop, buttons, checking data)?
an **as const array**, with the union worked out from it
- **enum** : recognise it in other people's code - you do not need it

Checking data from JSON

```
export const toDiet = (text: string): Diet => {  
  const diet = DIETS.find((value) => value === text);  
  if (diet === undefined) {  
    throw new Error(`"${text}" is not a diet`);  
  }  
  return diet;  
};
```

```
assertThrows(() => toDiet("pescatarian"), Error);
```

- JSON gives `diet: string` - the compiler will not accept it as a `Diet`
- try it: put `"vegen"` in `menu.json`

A static factory method

```
export class Dish {  
  constructor(  
    public readonly name: string,  
    public readonly course: Course,  
    public readonly diet: Diet,  
    public readonly priceInCents: number,  
  ) {}  
  
  public static fromData(data: DishData): Dish {  
    return new Dish(data.name, toCourse(data.course),  
      toDiet(data.diet), data.price);  
  }  
}
```

`menu.map((data) => Dish.fromData(data))` - no dish needed to call it

Project 3: Ticket Numbers

Ticket Numbers

Customers

Take a ticket

Your ticket is A004.

Waiting

A002A003A004

Counter

Now serving

A001

Serve next

Tickets issued today: 4

New day

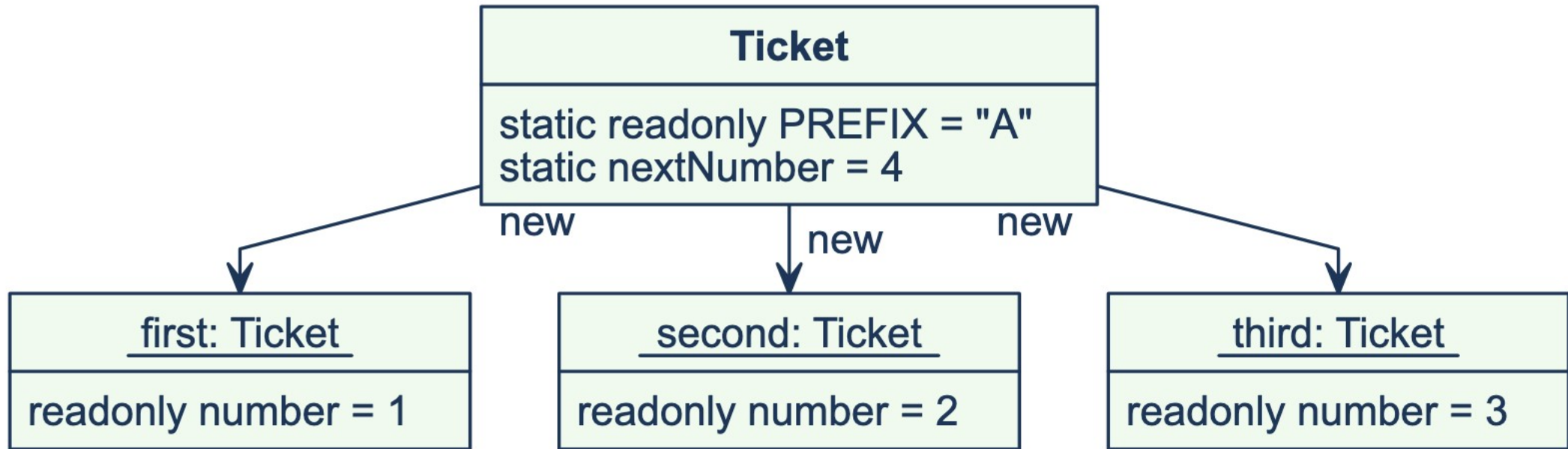
Where should "the next number" be stored?

A static counter

```
export class Ticket {  
  public static readonly PREFIX = "A";  
  private static nextNumber = 1;    // one, shared  
  
  public readonly number: number;  // one per ticket  
  
  constructor() {  
    this.number = Ticket.nextNumber;  
    Ticket.nextNumber++;  
  }  
}
```

Class and objects

static belongs to the class; everything else belongs to each object



`new Ticket()` takes `nextNumber`, then adds 1 to it. Use `Ticket.PREFIX` and `Ticket.issuedCount()` on the class, `first.number` on an object.

Static methods

```
public static issuedCount(): number {  
    return Ticket.nextNumber - 1;  
}
```

No object, so no `this.field`:

Property 'count' does not exist on type 'typeof Broken'.

Java allows `ticket.PREFIX`; TypeScript does not:

Property 'PREFIX' does not exist on type 'Ticket'. Did you mean to access the static member 'Ticket.PREFIX' instead?

Two right tests, one red

```
Deno.test("the first ticket is number 1", ...);  
Deno.test("each new ticket gets the next number", () => {  
  const first = new Ticket();  
  const second = new Ticket();  
  assertEquals(first.number, 1);  
  // ...  
});
```

```
not ok 2 - each new ticket gets the next number  
  - 2  
  + 1
```

Alone, it passes. The static counter carried over from test 1.

A fresh start in every test

```
public static resetNumbering(): void {  
    Ticket.nextNumber = 1;  
}
```

```
const freshQueue = (): TicketQueue => {  
    Ticket.resetNumbering();  
    return new TicketQueue();  
};
```

- every new object starts fresh; a static field does not
- every test that makes tickets resets first

When not to use static

- `static readonly` constants: always fine
- static methods using only their parameters: fine
- static fields that **change**: global state
- two desks, each numbering from 1? One static counter cannot do it
- ask: "**could there ever be two of these?**" - then use an object

Summary

- `const` / `readonly` / `static readonly`
- union types; `switch` with a return type and no `default`
- loop over the values to test every one
- `as const` array + `(typeof VALUES)[number]` : write values once
- `enum` works, but is unlike the rest of TypeScript
- check strings from outside: `find`, then throw
- static belongs to the class; reset static state in every test

Challenges

1. What should drivers do? (a switch, every colour)
2. A new diet - let the compiler find the switches
3. Back to the start after A999
4. Courses in order - `COURSES.indexOf`
5. Two desks - static counter to `TicketMachine`
6. A junction - at least one light red in **every** phase